# 第 3 章 神经网络与训练

> 本章目标：沿着"结构怎么进化"和"能力从哪来"两条线，把神经网络的发展脉络串起来，并且第一次真正接触数据、硬件、软件栈这些"模型之外、却决定成败"的东西。

## 3.1 从感知机到 MLP

**感知机（perceptron）**是最早的神经网络单元：给若干输入加权求和，超过阈值输出 1，否则输出 0。它可以学会"与""或"这类线性可分的判断。

它的天花板很硬：**连"异或"（XOR）都学不会。** 因为异或的答案没法用一条直线切开——这类问题叫线性不可分。

**多层感知机（MLP）**的解法很直接：把多个感知机叠起来，中间加非线性激活（ReLU 等），再堆几层。这样"直线切不开"的问题，可以靠多层折线去逼近。

> 要点：单层只能划直线，多层才能画出曲线。深度带来的不是"算力更多"，而是**表达能力的质变**。

理论上，足够宽的一层网络能逼近任意连续函数，这叫**万能逼近定理**。这句话常被过度解读——它只保证"存在这样的网络"，不保证"训练能找到它"。实践中，深层往往比又宽又浅更好学，所以大家才往深了堆。

## 3.2 CNN：为什么卷积特别适合图像

全连接层处理图片有个荒唐之处：一张图里相邻像素高度相关，而全连接层把每个像素和下一层的每个神经元都连起来，参数爆炸还学不到"局部图案"。

**卷积神经网络（CNN）**用三个设计解决了这件事：

1. **局部感受野**：每个神经元只看一小块区域。图像的本质就是局部特征——边缘、拐角、纹理——这一假设非常贴合现实。
2. **参数共享**：同一个卷积核扫遍全图。所以"猫耳朵"出现在左上角还是右下角，用的是同一组参数。
3. **层次堆叠**：浅层学边缘，中层学轮廓和部件，深层学整体物体。

这也解释了 CNN 的局限：它擅长处理"有局部结构、且平移不该改变含义"的数据。对文本、对长距离依赖，效果就没那么理想了。

## 3.3 RNN / LSTM：处理序列，以及它为什么不够

文本、语音、时间序列都是**有顺序**的。**循环神经网络（RNN）**的思路是：维护一个"记忆状态"，每读一个词就更新一次，把它传给下一步。

问题有两个：

- **梯度消失 / 爆炸**：信息要穿过很长的时间步才能传回，反复相乘之后信号不是消失就是爆炸。序列一长，网络就"记不住开头"。
- **无法并行**：第 t 步必须等第 t−1 步算完，训练时 GPU 大部分时间在空转。

**LSTM** 用"门"机制缓解了第一个问题——输入门、遗忘门、输出门决定哪些信息保留、哪些丢弃，让长距离依赖能传得更远。这在很长一段时间里是序列建模的主流。

但它没解决第二个问题。**序列长度一长、数据一多，串行计算就成了瓶颈。** 而 Transformer 恰好把这两件事一起解决了：既能让任意两个位置直接建立联系，又能整段并行计算。这部分留到第四章展开。

# 3.4 训练方式：能力是怎么"喂"进去的

| 方式 | 数据形态 | 一句话 | 典型任务 |
| --- | --- | --- | --- |
| 监督学习（supervised） | 输入 + 标注答案 | 有标准答案，照着纠错 | 图像分类、缺陷检测、语音识别 |
| 无监督学习（unsupervised） | 只有输入，无标签 | 自己找结构 | 聚类、降维、异常检测 |
| 自监督学习（self-supervised） | 从数据本身造标签 | 遮住一部分，让模型猜 | 大模型预训练的主要方式 |
| 半监督学习（semi-supervised） | 少量标注 + 大量无标注 | 用少量标签带动大量数据 | 标注昂贵的工业场景 |
| 对抗训练（adversarial） | 生成器 + 判别器互相博弈 | 造假的和鉴假的互相练 | 图像生成、对抗样本防御 |
| 强化学习（RL） | 动作 + 奖励信号 | 做对了给分，自己摸索 | 游戏、机器人控制、策略优化 |
| RLHF | 人类偏好排序 | 用人的喜好调行为风格 | 对齐（alignment） |

其中**自监督学习**值得单独讲一句：它不给人工标签，而是从数据中挖出题目——比如遮住一句话里的一部分让模型补全，或者预测下一个词。**大模型之所以能用上几乎整个互联网的文本，就是因为不需要人工标注，标签是数据自带的。** 这是"大模型时代"在方法论上的关键一步。

**RLHF（基于人类反馈的强化学习）**常被误解。它不负责教模型知识——知识在预训练阶段就学完了。它做的是**对齐**：让模型的回答更符合人的偏好，更有帮助、更少有害内容。用考试类比：预训练是"把课本都读了"，RLHF 是"请了一群阅卷老师，告诉你什么样的答卷更讨喜"。它能改变**风格与倾向**，改不了模型不知道的事实。

## 3.5 数据：模型的全部原料

**划分数据**是第一条纪律：训练集（train）用来学，验证集（val）用来调超参数，测试集（test）只在最后用一次。测试集一旦反复用来"改进"，它就不再是考试，而变成了另一份习题册。

其他关键事项：

- **清洗**：去重、去噪、修掉错标。重复样本会让模型死记，错标会让模型学歪。
- **标注**：最贵的一环。标注规范本身的质量，常常比算法选型更能决定结果。
- **数据投毒（poisoning）**：训练数据里混入恶意样本，会教会模型特定错误行为。来源不明的数据集要警惕。
- **数据偏见（bias）**：数据里有什么偏差，模型就学到什么偏差。缺某类人群、某类场景的样本，模型在这些地方就会系统性失败。

> 要点：**模型不会告诉你它没见过什么。** 它只会对没见过的输入给出一个自信但可能错误的答案。评估的第一件事，永远是问"训练数据覆盖了什么、漏了什么"。

## 3.6 硬件：为什么 GPU 成了标配

| | CPU | GPU |
|---|---|---|
| 擅长 | 少量复杂任务、分支判断、控制逻辑 | 海量简单计算并行 |
| 核心数 | 少而强 | 多而简单 |
| 在 AI 里的角色 | 调度、前后处理、小模型推理 | 训练与批量推理的主力 |

神经网络的核心运算是大规模矩阵乘加，天然适合并行，这正是 GPU 的强项。

生态上，**NVIDIA** 用 **CUDA** 建立了事实标准，几乎所有训练框架和加速库都优先支持它。**AMD** 走 ROCm，**Intel** 有 oneAPI 体系，**华为昇腾** 走 CANN 与自家指令集。跨厂商的通用图形接口有 **Vulkan**，llama.cpp 这类推理项目会用它做非 NVIDIA 卡的后端——**能跑，但性能与覆盖面通常不如 CUDA 顺**。

几个必须分清的概念：

- **显存容量**决定"能不能装下"——模型权重、中间激活、上下文缓存都要占地方。装不下就是装不下，没有技巧能绕过去。
- **显存带宽**决定"算得多快"——推理时数据要不停地在显存和计算单元之间搬运。很多小模型在消费级卡上慢，卡在带宽而不是算力。
- **集群与 NVLink / 超节点**：单卡装不下时，要把多张卡连起来。卡间互联速度直接决定训练效率，于是有了 NVLink 这类高带宽互联，以及把多卡组成"超节点"的做法。

**云端 vs 边缘**是另一组取舍：

| | 云端 | 边缘（本地设备） |
|---|---|---|
| 能力 | 能跑大模型 | 通常只能跑小模型 |
| 延迟 | 受网络影响 | 低且稳定 |
| 隐私 | 数据要出设备 | 数据不出设备 |
| 成本 | 按次付费 | 一次性投入 + 电费 |

机器人是典型的混合场景：**实时控制与避障必须在边缘**（迟 200 毫秒可能就撞了），**复杂规划与对话可以放云端**。

## 3.7 软件栈：会用比会造更重要

- **PyTorch / TensorFlow**：训练框架。当前研究与工程的主流是 PyTorch，TensorFlow 在既有生产系统里仍有存量。

- **训练框架 vs 推理引擎**：这是两个不同的东西。训练框架要能算梯度、能改参数；推理引擎只做前向计算，追求快、追求省显存。**llama.cpp、Ollama** 属于后者——它们让你在一台普通电脑上跑起量化后的大模型，但**不能用来训练**。
- **ONNX**：一种模型交换格式。训练完导出成 ONNX，可以脱离原框架在多种运行时上部署，便于跨平台和跨硬件。它提高可移植性，但不保证性能最优。

## 3.8 经典实验：用眼睛看现象

下面这些实验不要求你调出最优结果，要求你**观察现象并解释**。

| 实验 | 数据集与任务 | 学生能观察什么 |
| --- | --- | --- |
| 手写数字识别（CNN） | 手写数字图像 → 0–9 | 卷积层学到什么；训练 loss 与验证 loss 何时分叉 |
| 手写数字识别（SVM） | 同上，但用传统分类器 | 与 CNN 的差距有多大；小数据下谁更划算 |
| 垃圾邮件分类 | 邮件文本 → 正常/垃圾 | 精确率与召回率的取舍；误杀一封重要邮件的代价 |
| 房价预测 | 房屋特征 → 价格 | 回归任务的损失怎么定义；异常样本的破坏力 |
| K-means 聚类 | 无标签数据 → 若干簇 | 簇的数量要人定；初值敏感 |
| 电影推荐 | 用户-物品评分 → 推荐列表 | 冷启动问题；推荐结果为什么"越看越窄" |
| 异常检测 | 正常样本 → 标出异常 | 只有单类样本时怎么做；阈值怎么选 |
| 强化学习小游戏 | 环境 + 奖励 → 策略 | 奖励设计错了会学出什么怪行为；探索与利用的矛盾 |

# 3.9 强化学习与搜索式 AI

**强化学习（RL）**的三件套是：**奖励（reward）**告诉它什么算好，**探索（exploration）**让它敢试没试过的动作，**策略（policy）**是它学到的"看到什么就做什么"。

> 类比：训练一只狗。"坐下"做对了给零食（奖励），做错了没反应；它会试探各种动作（探索），最后固化成一套反应习惯（策略）。

难点在于**奖励设计**。奖励给错了，模型会精确地优化出你没想到的怪行为——这叫奖励黑客。

另一条完全不同的路线是**搜索式 AI**：**AlphaGo** 这类系统把"策略网络"和"蒙特卡洛树搜索"结合，同时用到了**极大极小（minimax）**与 **Alpha-Beta 剪枝**这类经典博弈搜索思想。这提醒我们：**现代强 AI 常常是"学习"与"搜索"的混合体，不是单纯某一边的胜利。**

# 课后题

1. （概念）用一句话说明为什么单层感知机学不会异或，以及 MLP 是怎么解决的。
2. （概念）列出 CNN 适合图像的三条理由，并指出它对哪类数据不合适。
3. （概念）自监督学习和监督学习的根本区别是什么？为什么大模型离不开自监督？
4. （动手，无需本地环境）打开浏览器里的 Teachable Machine 一类工具，用摄像头采集两类手势各若干张，训练一个分类器。然后：① 只给每类很少的样本，观察结果；② 让测试时的光线与训练时明显不同，观察准确率变化。记录并解释你看到的现象。
5. （动手，无需本地环境）设想你要做"零件缺陷检测"：写出你会如何划分训练/验证/测试集，如何避免同一批次的图片同时出现在训练集和测试集里，以及为什么这个细节很重要。
6. （思考）如果给强化学习智能体的奖励是"移动距离越远越好"，它可能学出什么你不想看到的行为？这个例子说明了奖励设计的什么困难？
7. （思考）有人说"有了大模型就不需要小模型了"。结合 3.6 的云端/边缘取舍和 3.1–3.3 的结构演化，谈谈你的看法。